

TP 1

Nassim EL HADDAD

Exercise 1

1.c

```
Wed Sep 16 13:45:33 2026
+-----+
| NVIDIA-SMI 595.84                Driver Version: 595.84          CUDA Version: 13.2     |
+-----+-----+
| GPU   Name      Perf      Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp  Perf    Pwr:Usage/Cap |         Memory-Usage | GPU-Util  Compute M. |
|=====+=====+
| 0 NVIDIA L4      P8                Off      | 00000000:06:10.0 Off |           0         |
| N/A   33C    P8         16W /  72W |  3MiB / 23034MiB |      0%    Default  |
+-----+-----+
+-----+
| Processes:                        |
| GPU   GI    CI          PID    Type    Process name          GPU Memory |
| ID   ID     ID              |                 |           Usage     |
+-----+-----+
| No running processes found      |
+-----+
```

-> C'est le GPU Nvidia L4 qui m'est alloué (23Go à peu près de mémoire)

1.d

```
[compute] [job 1548] nelhaddad@starfighter-slurm-node-04-1:~
$ squeue -u $USER
          JOBID PARTITION     NAME     USER ST       TIME  NODES MODELIST(REASON)
          1548      gpu      bash nelhadda  R        2:40        1 starfighter-slurm-node-04-1
```

-> La commande pour tuer le JobID est la suivante : scancel 1548

1.f



```
#!/bin/bash
#SBATCH --partition=gpu
#SBATCH -t 01:00:00
#SBATCH --gres=gpu:1
#SBATCH --cpus-per-task=1
#SBATCH --mem=8G
#SBATCH -J hello-slurm
#SBATCH -o Logs/%x-%j.out
#SBATCH -e Logs/%x-%j.err

set -euo pipefail
mkdir -p logs

echo "Job $SLURM_JOB_ID on $SLURM_NODELIST"
nvidia-smi || echo "nvidia-smi indisponible"
echo "Bonjour depuis SLURM !"
```

```
[compute] [job 1552] nelhaddad@starfighter-slurm-node-01-1:~/logs
$ ls
❖ hello-slurm-1553.err  ❖ hello-slurm-1553.out
```

-> Le fichier de log généré est dans le dossier /logs et se nomme hello-slurm-1553.out (il y a également un .err dans le dossier)

1.f

```
$ sacct -j 1552 --format=JobID,State,Elapsed,MaxRSS,ReqMem,ReqCPUS
JobID          State      Elapsed      MaxRSS      ReqMem      ReqCPUS
-----
1552           CANCELLED+  00:03:30          8G           1
1552.extern    CANCELLED  00:03:30          1           1
1552.0         CANCELLED  00:03:30          1           1
```

-> ReqMem correspond à la RAM demandée par le job, tandis que MaxRSS correspond à la quantité maximale de RAM réellement utilisée pendant son exécution.

Exercice 2

2.a

```
[compute] [job 1552] nelhaddad@starfighter-slurm-node-01-1:~/logs
$ srun: error: Timed out waiting for job step to complete
[controller] nelhaddad@arcadia-slurm-controller:~
$ srun --partition=gpu --gres=gpu:1 --time=01:00:00 --cpus-per-task=1 --mem=8G --pty bash
[compute] [job 1554] nelhaddad@starfighter-slurm-node-01-1:~
$ curl -L -O "https://github.com/conda-forge/miniforge/releases/latest/download/Miniforge3-$(uname)-$(uname -m).sh"
bash Miniforge3-$(uname)-$(uname -m).sh
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left     Speed
  0     0    0     0    0     0      0      0      0     0
  0     0    0     0    0     0      0      0      0     0
100 118.7M 100 118.7M    0     0  18.48M      0    00:06    00:06    19.85M

Welcome to Miniforge3 26.7.2-0

In order to continue the installation process, please review the license
agreement.
Please, press ENTER to continue
```

2.b

Ce cluster est réservé à l'enseignement.

```
#####  
[controller] nelhaddad@arcadia-slurm-controller:~  
$ mamba  
Version: 2.9.0
```

```
[controller] nelhaddad@arcadia-slurm-controller:~  
$ mamba create -n deeplearning python=3.10  
Fetch Shard Index for conda-forge/linux-64 ✓Done (0.3 sec)  
Fetch Shard Index for conda-forge/noarch ✓Done (0.2 sec)  
Fetching and Parsing Packages' Shards ✓Done (7.0 sec)  
  
Resolving Environment ✓Done (0.1 sec)  
  
Transaction  
  
Prefix: /mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning  
  
Updating specs:  
  
- python=3.10  
- pip  
  
Package Version Build Channel Size  
-----  
Install:  
  
+ _openmp_mutex 4.5 20_gnu conda-forge Cached  
+ bzip2 1.0.8 hda65f42_10 conda-forge Cached  
+ ca-certificates 2026.7.22 hbd8a1cb_0 conda-forge Cached  
+ icu 78.3 py310h44b86e0_2 conda-forge Cached  
+ ld_impl_linux-64 2.46.1 default_hbd61a6d_102 conda-forge Cached  
+ libexpat 2.8.1 hecca717_1 conda-forge Cached  
+ libffi 3.7.0 h81df57d_1 conda-forge Cached  
+ libgcc 16.2.0 ha9f2e26_5 conda-forge 1MB  
+ libgomp 16.2.0 he0feb66_5 conda-forge 642kB  
+ liblzma 5.8.3 hb03c661_1 conda-forge Cached  
+ libns1 2.0.1 hb9d3cd8_1 conda-forge 34kB  
+ libsqlite 3.53.4 h13e7031_1 conda-forge Cached
```

```

Linking zstd-1.5.7-hb78ec9c_7
Linking _openmp_mutex-4.5-20_gnu
Linking ld_impl_linux-64-2.46.1-default_hbd61a6d_102
Linking libgcc-16.2.0-ha9f2e26_5
Linking libstdc++-16.2.0-h934c35e_5
Linking bzip2-1.0.8-hda65f42_10
Linking libnscl-2.0.1-hb9d3cd8_1
Linking libexpat-2.8.1-hecca717_1
Linking libuuid-2.42.3-hcfc3c73_0
Linking libffi-3.7.0-h81df57d_1
Linking liblzma-5.8.3-hb03c661_1
Linking ncurses-6.6-hdb14827_1
Linking tk-8.6.13-noxft_h1df4ec4_4
Linking libxcrypt-4.4.38-h280c20c_0
Linking icu-78.3-py310h44b86e0_2
Linking readline-8.3-hd6e31c0_1
Linking libsqlite-3.53.4-h13e7031_1
Linking tzdata-2026c-h151e31d_0
Linking ca-certificates-2026.7.22-hbd8a1cb_0
Linking openssl-3.6.4-h781a0a9_0
Linking python-3.10.21-h267e890_0_cpython
Linking packaging-26.3-pyh332efcf_0
Linking setuptools-84.0.0-pyh332efcf_0
Linking wheel-0.48.0-pyhd8ed1ab_0
Linking pip-26.2.1-pyh8b19718_0

```

Transaction finished

To activate this environment, use:

```
mamba activate deeplearning
```

Or to execute a single command in this environment, use:

```
mamba run -n deeplearning mycommand
```

```

[controller] nelhaddad@arcadia-slurm-controller:~
$ mamba activate deeplearning
[controller] nelhaddad@arcadia-slurm-controller:~
$ |

```

-> La commande est la suivante : `python --version && which python`

`python -version` : vérifier la version de python que l'on a dans l'environnement virtuel (ici 3.10.21)

`which python` : vérifier le chemin du binaire utilisé, ici

`/mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning/bin/python`

```

[controller] nelhaddad@arcadia-slurm-controller:~
$ python --version && which python
Python 3.10.21
/mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning/bin/python

```

2.c

```

[controller] nelhaddad@arcadia-slurm-controller:~
$ mamba install pytorch torchvision torchaudio pytorch-cuda=12.1 -c pytorch -c nvidia
mamba install tensorboard -c conda-forge
Using Cached Shard Index for conda-forge/linux-64 ✓Done
Using Cached Shard Index for conda-forge/noarch ✓Done
Fetching and Parsing Packages' Shards ✓Done (48.9 sec)
Fetch Shard Index for pytorch/linux-64 ✓Done (0.2 sec)
Fetch Shard Index for pytorch/noarch ✓Done (0.1 sec)
Fetching and Parsing Packages' Shards ✓Done (0.5 sec)
Fetch Shard Index for nvidia/linux-64 ✓Done (0.2 sec)
Fetch Shard Index for nvidia/noarch ✓Done (0.1 sec)
Fetching and Parsing Packages' Shards ✓Done (8.0 sec)
Using Cached Shard Index for conda-forge/linux-64 ✓Done
Using Cached Shard Index for conda-forge/noarch ✓Done
Fetching and Parsing Packages' Shards ✓Done (8.2 sec)
Using Cached Shard Index for pytorch/linux-64 ✓Done
Using Cached Shard Index for pytorch/noarch ✓Done
Fetching and Parsing Packages' Shards ✓Done (0.0 sec)
Using Cached Shard Index for nvidia/linux-64 ✓Done
Using Cached Shard Index for nvidia/noarch ✓Done
Fetching and Parsing Packages' Shards ✓Done (0.0 sec)

Pinned packages:

- python=3.10

Resolving Environment ✓Done (1.7 sec)

Transaction

Prefix: /mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning

Updating specs:

- pytorch
- torchvision
- torchaudio
- pytorch-cuda=12.1

```

```

Change:
-----
- _openmp_mutex          4.5  20_gnu          conda-forge  Cached
+ _openmp_mutex          4.5  8_kmp_llvm        conda-forge      8kB

Downgrade:
-----
- setuptools             84.0.0 pyh332efcf_0      conda-forge  Cached
+ setuptools             81.0.0 pyh332efcf_0      conda-forge  677kB

Summary:

Install: 189 packages
Change: 1 packages
Downgrade: 1 packages

Total download: 1GB

```

```

warning  libmamba Security Warning: This transaction includes executing package scripts (pre/post-link/unlink) if present. These scripts can contain arbitrary code. Please ensure you trust the package sources.
Confirm changes: [Y/n] Y

Transaction starting
protobuf          410.9kB @ 146.4kB/s  0.0s
grpcio            1.0MB @ 3.5MB/s  0.1s
werkzeug          258.2kB @ 1.3MB/s  0.2s
libre2-11         210.6kB @ 1.2MB/s  0.2s
absl-py           115.2kB @ 781.8kB/s  0.2s
markdown          86.7kB @ 1.1MB/s  0.1s
re2               27.4kB @ 265.6kB/s  0.2s
tensorboard-data-server 3.5MB @ 3.8MB/s  0.9s
tensorboard       5.3MB @ 5.3MB/s  1.0s
libgrpc           7.4MB @ 5.3MB/s  1.1s
Linking protobuf-7.35.1-py310hffa8988_1
Linking libre2-11-2025.11.05-h60473fc_2
Linking tensorboard-data-server-0.7.0-py310hf0f1799_4
Linking c-ares-1.24.0-hebe6cf0_2
Linking re2-2025.11.05-h90463f1_2
Linking libgrpc-1.83.1-h284dbb2_0
Linking grpcio-1.83.1-py310h89ca40a_0
Linking werkzeug-3.1.0-pyhcf101f3_0
Linking absl-py-2.5.0-pyhd8ed1ab_0
Linking zipp-4.1.0-pyhcf101f3_0
Linking importlib-metadata-9.0.1-pyhcf101f3_0
Linking markdown-3.10.3-pyhcf101f3_0
Linking tensorboard-2.20.0-pyhe01879c_1
Transaction finished

```

2.d

```

python

import torch

print("PyTorch version:", torch.__version__)

gpu_available = torch.cuda.is_available()
print("CUDA available:", gpu_available)

if gpu_available:
    print("Device count:",
          torch.cuda.device_count())
    print("Device 0 name:",
          torch.cuda.get_device_name(0))
else:
    print("Attention, aucun GPU détecté !")

```

```
[controller] nelhaddad@arcadia-slurm-controller:~  
$ python check_gpu.py  
PyTorch version: 2.13.0  
CUDA available: False  
Attention, aucun GPU détecté !
```

-> Premièrement, PyTorch peut être installé sans support CUDA, avec une version uniquement compatible avec le CPU.

Deuxièmement, le GPU peut ne pas être accessible au job, par exemple s'il n'a pas été demandé avec SLURM ou si les pilotes NVIDIA/CUDA ne sont pas disponibles sur le nœud.

```
[controller] nelhaddad@arcadia-slurm-controller:~  
$ cat environment.yml  
name: deeplearning  
channels:  
  - conda-forge  
  - pytorch  
dependencies:  
  - pip  
  - python=3.10  
  - pytorch  
  - pytorch-cuda=12.1  
  - tensorboard  
  - torchaudio  
  - torchvision  
  
prefix: "/mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning"
```

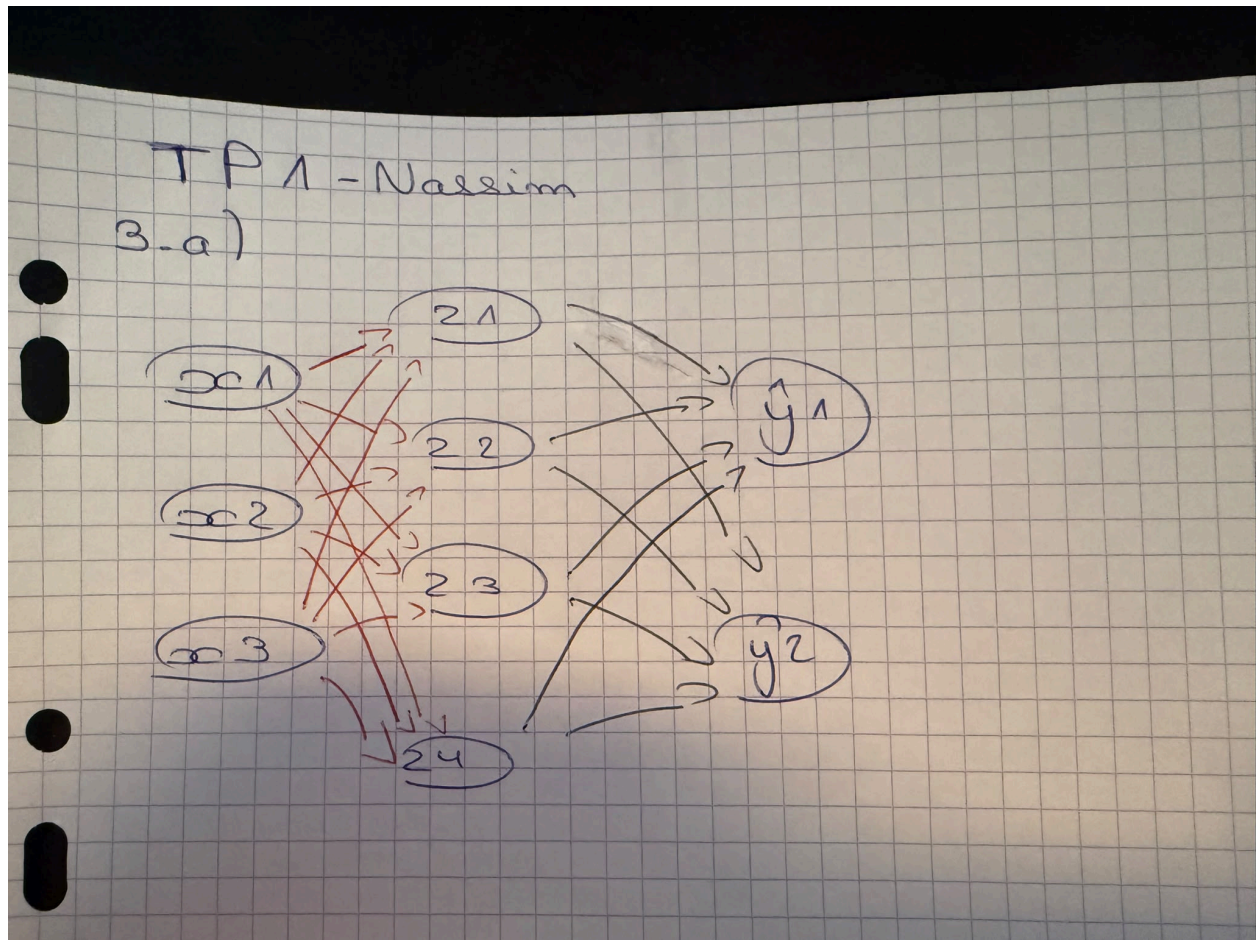
2.f

-> La commande est la suivante : `tensorboard --version`

Elle permet de connaître la version de tensorboard que l'on a.

Exercice 3

3.a



3.b

$$Z = X \cdot W_1^T + b_1$$

$$H = \text{ReLU}(X \cdot W_1^T + b_1) = \text{ReLU}(Z)$$

$$Y = H \cdot W_2^T + b_2 = \text{ReLU}(Z) \cdot W_2^T + b_2 = \text{ReLU}(X \cdot W_1^T + b_1) \cdot W_2^T + b_2$$

Dimensions :

$X : (N, 3)$

$W_1 : (4, 3)$

$b_1 : (1, 4) \rightarrow \text{diffusé en } (N, 4)$

$H : (N, 4)$

$W_2 : (2, 4)$

$b_2 : (1, 2) \rightarrow \text{diffusé en } (N, 2)$

$Y : (N, 2)$

3.c

3.c

$\rightarrow$ La valeur de f est 0,5

- $f = q + 3$ donc $\frac{\partial f}{\partial q} = 1$ et $\frac{\partial f}{\partial 3} = 1$
- $q = x/y$ donc $\frac{\partial q}{\partial x} = \frac{1}{y}$ et $\frac{\partial q}{\partial y} = -\frac{x}{y^2}$
- $\frac{\partial f}{\partial x} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial x} = 1 \times \frac{1}{y} = \frac{1}{y}$
- $\frac{\partial f}{\partial y} = \frac{\partial f}{\partial q} \frac{\partial q}{\partial y} = 1 \times -\frac{x}{y^2} = -\frac{x}{y^2}$
- Ainsi $\rightarrow \frac{\partial f}{\partial x} = \frac{1}{y}$, $\frac{\partial f}{\partial y} = -\frac{x}{y^2}$ et $\frac{\partial f}{\partial 3} = 1$
- $x=2, y=4$ et $3=0$: $\frac{\partial f}{\partial x} = \frac{1}{4}$, $\frac{\partial f}{\partial y} = -\frac{2}{16} = -\frac{1}{8}$, $\frac{\partial f}{\partial 3} = 1$

3.d

3.d)

$$\theta' = \theta - m \frac{\partial f}{\partial \theta}$$

pour x on a :

$$x' = x - m \frac{\partial f}{\partial x} = 2 - 1 \times 0,25 = 1,75$$

pour y on a :

$$y' = y - m \frac{\partial f}{\partial y} = 4 - 1 \times 0,125 = 3,875$$

pour z on a :

$$z' = z - m \frac{\partial f}{\partial z} = 0 - 1 \times 1 = -1$$

Donc la sortie est :

$$f' = f(x', y', z') = \frac{x'}{y'} + z'$$

$$f' = \frac{1,75}{3,875} - 1 = -0,52$$

Ainsi, $f(x, y, z) = 0,5 > f(x', y', z')$ donc la valeur a diminué comme attendu

3.e

Pourquoi utilisons-nous la règle de la chaîne (chain rule) pour calculer les gradients dans les réseaux de neurones profonds ?

-> La règle de la chaîne permet de calculer le gradient d'une fonction composée de plusieurs couches en combinant les gradients locaux de chaque opération. Elle permet ainsi de propager l'erreur depuis la sortie du réseau jusqu'aux couches précédentes afin de mettre à jour les poids.

Quelles sont les principales raisons d'utiliser des mini-batches plutôt que d'optimiser sur un seul exemple à la fois ou sur l'ensemble total des données ?

-> Les mini-batches offrent un compromis entre la rapidité du calcul sur un seul exemple et la stabilité du calcul sur l'ensemble des données. Ils permettent de réduire l'utilisation de la

mémoire, d'accélérer les calculs grâce à la vectorisation et de produire des mises à jour plus stables que l'utilisation d'un seul exemple.

3.f

Tâche	Fonction finale (Sortie)	Fonction de perte (Loss)
Classification binaire	1. Sigmoidé	A. Binary Cross-Entropy (ou Entropie croisée binaire)
Classification multi	2. Softmax	B. Cross-Entropy (ou Entropie croisée)
Régression pure	3. Identité (aucune)	C. MSE (Mean Squared Error)

Exercice 4

4.a

-> `batch_size=32` signifie que les données sont traitées par groupes de 32 images avant chaque mise à jour des poids. Cela permet de mieux gérer l'utilisation de la mémoire tout en accélérant les calculs.

`shuffle=True` mélange les données à chaque époque pendant l'entraînement afin d'éviter que le modèle apprenne un ordre particulier des exemples et ainsi améliorer sa généralisation. Pour le test, `shuffle=False` conserve l'ordre des données, ce qui facilite une évaluation reproductible.

4.b

1. Pourquoi utilise-t-on `torch.flatten(x, 1)` dans la méthode `forward` ?

Les images CIFAR-10 ont une forme `(3, 32, 32)` correspondant aux 3 canaux RGB et aux dimensions `32×32` pixels. Une couche linéaire (`nn.Linear`) attend un vecteur en entrée. `torch.flatten(x, 1)` conserve la dimension du batch et aplati toutes les dimensions

suivantes. Ainsi, chaque image passe de $(3, 32, 32)$ à un vecteur de **3072 valeurs** ($3 \times 32 \times 32 = 3072$), compatible avec `nn.Linear(3072, 128)`.

2. Pourquoi ne faut-il pas ajouter de Softmax avant `nn.CrossEntropyLoss` ?

Dans PyTorch, `nn.CrossEntropyLoss` attend directement les **logits bruts** produits par la dernière couche du réseau. Elle applique elle-même les opérations nécessaires, notamment le calcul du log-softmax, de manière plus stable numériquement. Ajouter une fonction `Softmax` avant `CrossEntropyLoss` est donc inutile et peut dégrader la stabilité numérique et l'apprentissage. Il faut donc simplement retourner les logits de la dernière couche.

4.c

La différence est que `optimizer.zero_grad()` efface les gradients du batch précédent, tandis que `loss.backward()` calcule les nouveaux gradients à partir de la perte. On remet donc d'abord les gradients à zéro, puis on calcule ceux du batch actuel pour pouvoir ensuite mettre à jour les paramètres du modèle.

```
$ srun --partition=gpu --gres=gpu:1 --time=01:00:00 --cpus-per-task=1 --mem=8G --pty python train.py
Using device: cpu
100.0%
/mnt/hdd/homes/nelhaddad/miniforge3/envs/deeplearning/lib/python3.10/site-packages/torch/utils/data/dataloader.py:1102:
UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.
  super().__init__(loader)
Epoch 01 | loss=2.0940 | acc=0.3293
Epoch 02 | loss=2.1155 | acc=0.3567
Epoch 03 | loss=2.1447 | acc=0.3606
Epoch 04 | loss=2.0691 | acc=0.3842
Epoch 05 | loss=2.0697 | acc=0.3882
Epoch 06 | loss=2.0513 | acc=0.3950
Epoch 07 | loss=2.0096 | acc=0.4057
Epoch 08 | loss=1.9838 | acc=0.4151
Epoch 09 | loss=1.9656 | acc=0.4211
Epoch 10 | loss=1.9489 | acc=0.4266
```

4.d

1. Pourquoi utilise-t-on `torch.no_grad()` : lors de l'évaluation ?

Lors de l'évaluation, les poids du modèle ne sont pas modifiés, donc il n'est pas nécessaire de calculer les gradients. `torch.no_grad()` permet de désactiver ces calculs, ce qui réduit l'utilisation de la mémoire et le nombre de calculs nécessaires. L'évaluation est ainsi plus rapide et consomme moins de ressources.

2. Quelle accuracy attendre pour des prédictions aléatoires sur CIFAR-10 ?

CIFAR-10 contient 10 classes. Si le classificateur choisit une classe complètement au hasard avec une probabilité identique pour chaque classe, la probabilité d'obtenir la bonne réponse est de :

1 / 10 = 10 % On devrait donc s'attendre à une accuracy d'environ **10 %** sur le jeu de test.

Exercice 5

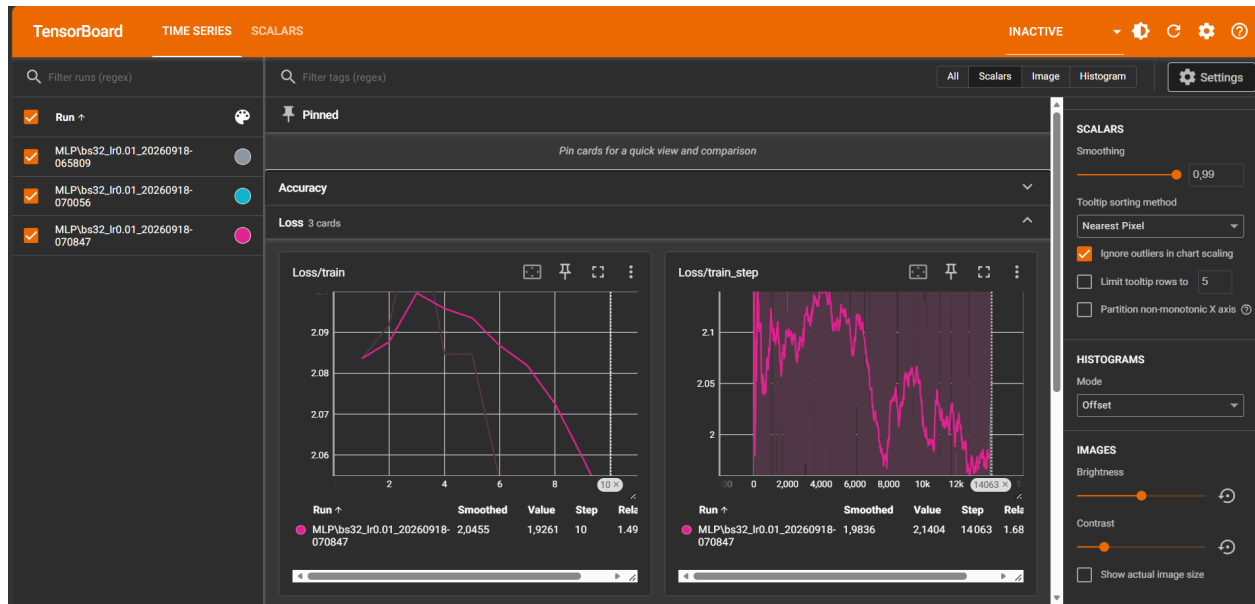
5.a

Le nom du dossier de logs (`run_name`) doit contenir la date, l'heure et les principaux hyperparamètres de l'expérience, comme la taille du batch (`bs`) et le learning rate (`lr`). Cela permet de distinguer facilement les différentes expériences, de retrouver leurs résultats et de garder une organisation claire des logs sans risque de les confondre ou de les écraser.

5.d

J'ai réglé le smoothing à 0,98. Une valeur entre 0,95 et 0,98 permet de bien faire ressortir la tendance générale tout en gardant les variations importantes.

La courbe `Loss/train_step` est plus bruitée car elle représente la loss calculée à chaque mini-batch. Comme les données changent d'un mini-batch à l'autre, la loss peut varier fortement. À l'inverse, `Loss/train` correspond à une moyenne sur l'ensemble des mini-batches de l'époque, ce qui donne une courbe naturellement plus stable.



5.e

On remarque que le run (bs=128, lr=0.1) se distingue clairement des deux autres. Sa Loss/train diminue progressivement, passant d'environ 1,8 à 1,4, tandis que sa Loss/val reste autour de 1,7. Cela montre que le modèle apprend de manière plus efficace et obtient de meilleurs résultats sur les données de validation. À l'inverse, pour les runs (bs=32, lr=0.001) et (bs=32, lr=0.01), la Loss/train reste proche de 2,0, avec une Loss/val autour de 2,1, ce qui indique un apprentissage moins efficace.

Pour la courbe Accuracy/val, le run (bs=128, lr=0.1) atteint également la meilleure précision, avec environ 46 % à la dernière époque, contre environ 35 à 36 % pour les deux autres runs.

On peut identifier visuellement un phénomène de sur-apprentissage lorsque la Loss/train continue de diminuer alors que la Loss/val, après avoir diminué dans un premier temps, commence à augmenter. Le modèle devient alors de plus en plus performant sur les données d'entraînement, mais généralise moins bien sur de nouvelles données. Il s'adapte trop aux exemples d'entraînement, ce qui entraîne une augmentation de l'erreur lors de la validation.
